555F

AVIS IMPORTANT AUX ETUDIANTS

1. Chacune des feuilles de votre copie doit comporter une étiquette code à barres placée à l'endroit indiqué «coller ici votre code à barres».
2. Une copie d'examen comporte une seule «feuille principale» et des «feuilles suites». Sur chacune de vos feuilles, le code à barres est obligatoire.
3. Cette feuille d'examen est strictement personnelle. Elle ne doit comporter aucun signe distinctif. Elle doit être écrite en noir et/ou bleu.
4. Le non respect de l'une de ces recommandations peut faire attribuer la note ZERO à l'épreuve.

esprit Se former autrement
HONORIS UNITED UNIVERSITIES

NOTE

Coller ici votre code à barre

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20

00	25	50	75

Module : Techniques de Conteneurisation	Documents autorisés : OUI ☐ NON ☐
Enseignant(s) : UP Système	Calculatrice autorisée : OUI ☐ NON ☐
Classe(s) : 4Arctic, 4Iosys & 4Sleam	Internet autorisée : OUI ☐ NON ☐
Session: Principale	Nombre de page : 7.
Date : 25/05/2024 Heure: 11h00	Durée : 1h30

Questions de réflexion (4 pts) :

1. Comment Docker utilise-t-il les namespaces et cgroups pour isoler les conteneurs ? (1pt)

Les **namespaces** et les **cgroups** permettent d'exécuter plusieurs conteneurs indépendamment sur un seul hôte, en partageant le noyau tout en garantissant que chaque conteneur dispose de ses propres ressources et ne puisse pas interférer avec les autres → créant des espaces de noms virtuels pour différentes ressources système (user, PID, réseau, FS, IPC..)

2. Citer les trois principaux espaces de noms (ou repositories) des images Docker ? (0.75pt)

Images officielles (root-like images)
Images utilisateurs (user and organisations images)
Images auto-hébergées (self-hosted)

3. Dans un Dockerfile, afin de copier un fichier ou un répertoire de l'hôte vers une image Docker, on peut utiliser soit l'instruction COPY ou ADD. Expliquer la différence entre ces deux instructions. (0.5pt)

COPY: Accepte uniquement les sources locales sur le système hôte/ Exige un chemin de destination absolu
ADD: Accepte les sources locales et les URL distantes/ Prend en charge les chemins de destination relatifs et absolus.

Ne rien écrire ici

4. Quelle est la principale différence entre le « User-defined bridge network » et le « Default bridge network » ? (0.5pt)

« User-defined bridge network » permettre la résolution DNS des noms d'hôtes des conteneurs avec les adresses IP.

5. Dans le fichier « *docker-compose.yml* » ci-dessous, quel est l'ordre d'exécution des différents services ? (0.5pt)

```yaml
services:
  web:
    image: nginx
    depends_on:
      - db
      - redis
  db:
    image: postgres
  redis:
    image: redis
```

L'ordre d'exécution : db -- redis – web

6. Selon le fichier « *docker-compose.yml* » ci-dessous, les services « *web* » et « *app* » peuvent-ils communiquer entre eux ? Justifiez votre réponse. (0.75pt)

```yaml
version: '3'
services:
  web:
    build: ./web
    networks:
      - frontend
    depends_on:
      - app
  app:
    build: ./app
    networks:
      - backend
    volumes:
      - app-data:/var/lib/app-data
    environment:
      - APP_MODE=production
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
volumes:
  app-data:
```

Non. Le service « web » ne peut pas communiquer avec le service « app » car ils sont sur des réseaux différents (backend et frontend).

<u>Exercice</u> (6 pts)

Vous disposez d'une application composée d'un frontend développé avec « Angular » et d'une base de données « *MySQL* ». On souhaite créer une image Docker pour la partie frontend.

1. Quelles sont les différentes méthodes qu'on peut utiliser pour créer une image Docker (0.75 pt)

Créer une image : from scratch (0.25pt), docker build un dockerfile (0.25pt), ou docker Commit un conteneur (0.25pt),

2. Pour créer l'image du frontend, on a utilisé le Dockerfile ci-dessous. Compléter les champs manquants (2 pts)

0.25pt pour chaque instruction

```
# Utilisez une image de base adaptée pour Angular
FROM node:14 as build

# Copiez le contenu de l'application dans le conteneur
COPY . /app

# Déclarez le volume pour les données de l'application
VOLUME /app/data

# Installez les dépendances et construisez l'application Angular

# Ajoutez ici les instructions pour installer Node.js et Angular CLI, puis construire l'application
RUN npm install -g @angular/cli && \

npm install && \

ng build --prod

# Utilisez une image légère pour exécuter l'application Angular construite
FROM nginx:alpine

# Copiez les fichiers construits de l'application Angular dans le répertoire NGINX
COPY --from=build /app/dist /usr/share/nginx/html

# Exposez le port 80 pour accéder à l'application Angular
EXPOSE 80
```

3. Dans ce Dockerfile, quel principe est utilisé pour construire l'image de l'application Angular ? (0.5 pt)

Multi-stage build

4. Donner la ligne de commande appropriée pour créer l'image du frontend nommée « **angular-frontend »** (0.75 pt)

docker build -t angular-frontend **.**

0.25pt 0.25pt 0.25pt

5. Pour déployer l'application Angular, exécuter un conteneur nommé « *angular-container* ». (0.75pt)

docker run --name angular-container angular-frontend

0.25pt 0.25pt 0.25pt

6. Lorsque vous exécutez la base de données dans un conteneur Docker, vous avez remarqué que toutes les données de la base de données sont perdues à chaque redémarrage ou suppression du conteneur.

a. Dans Docker, quelles méthodes utilise-t-on pour rendre les données d'un conteneur persistantes ? (0.5 pt)

Créer des : ***Volumes de données***

Ou avec ***Bind Mount*** *(stockage dans des répertoires sur la machine hôte)*

b. On souhaite exécuter un conteneur de base de données (utilisant l'*image mysql:latest*) nommé **« db »,** tout en partageant le volume de données avec le conteneur « angular-container ». Donner la ligne de commande correspondante. (0.75pt)

docker run --name db --volumes-from angular-container mysql:latest

0.25pt 0.25pt 0.25pt

<u>Problème : (10 pts)</u>

Dans le cadre de déploiement d'une application basée sur les **microservices**, une entreprise a adopté la technologie de conteneurisation avec docker.

1. Expliquez la raison de ce choix (1 pt)

Portabilité et reproductibilité/ Flexibilité et évolutivité/ Isolation et empaquetage dans des conteneurs facilite leur déploiement et leur gestion ….

Cette application est basée sur trois services définis comme suivant :

- **Backend :** Une API RESTful développée en Node.js avec Express.
- **Base de Données :** Une base de données « *PostgreSQL* » pour stocker les données de l'application.

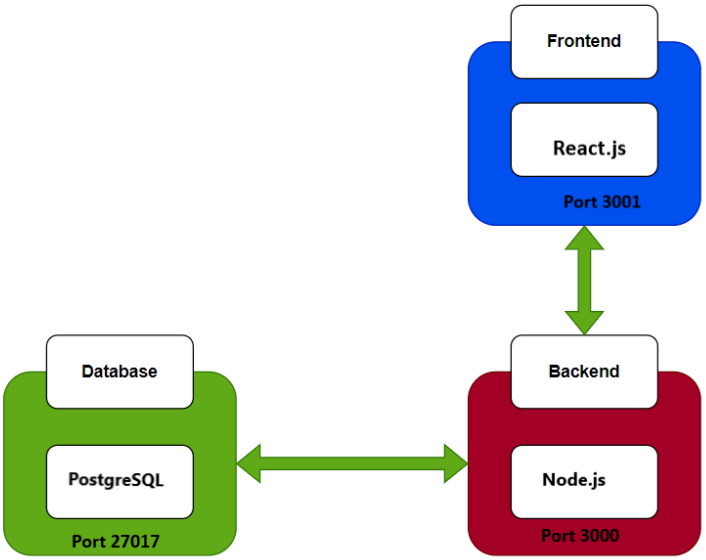
Deux variables d'environnement doivent être définies par ce service :

- ✓ « *POSTGRES_USER* »
- ✓ « *POSTGRES_PASSWORD* »

- **Frontend :** Une application frontend développée en React.js qui consomme l'API du backend.

SUITE

Coller ici votre
code à barre



Voici ci-dessous l'organisation de ce projet :

```
myapp/
│
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js
│   └── ...
│
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── src/
│   └── ...
│
└──
```

2. Pour orchestrer ces services sur votre serveur local, quel outil Docker doit on utiliser ? et dans quel fichier doit-on les configurer ? (1pt)

   Docker compose (0.5pt)
   docker-compose.yml (0.5pt)

3. Compléter le fichier de déploiement de cette application ? (3 pts)

```
version: '3'

services:
  backend:
    build: myapp/backend
    ports:
      - "3000 : 3000"
    depends_on:
      - database

  frontend:
    build: myapp/frontend
    ports:
      - "3001 : 3001"

    depends_on :

      - backend


  database:
    image: postgres      (accepter aussi postgres:13-alpine utilisé dans la suite de l'exercice)
    ports:
      - "27017: 27017"
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_USER: user
```

4. Une fois le fichier édité, quelle commande doit-on utiliser pour démarrer l'application ? (0.5pt)

   docker-compose up

5. Comment vérifier l'état d'exécution de cette application ? (0.5pt)

   docker-compose ps

6. Suite à une demande excessive de cette application, vous avez remarqué une dégradation des performances de votre serveur local. Proposer une solution pour déployer cette application sur plusieurs machines. (0.5pt)

   Utiliser Docker Swarm

7. Notre application sera déployée sur un cluster composé de 3 machines : machine A (master node), machines B et C (Worker nodes). Préciser les étapes à suivre pour mettre en place ce cluster. (0.75 pt)

- ***Initialisation du Swarm*** *dans la machine A (docker swarm init ..) (0.25pt)*
- ***Ajout de Workers*** *(machine B et C) : Une fois le Swarm initialisé, vous obtenez une commande* ***join*** *(0.25pt) pour ajouter des workers avec* ***un token*** *(0.25pt) (docker swarm join --token worker-token …)*

8. Pour le déploiement du « *stack* » des services de notre application sur ce cluster, on a utilisé le même fichier de la « ***Question 3*** ». Donner la commande adéquate. (0.75 pt)

docker stack deploy -c docker-compose.yml myapp

ou *: docker stack deploy --compose-file=docker-compose.yml myapp*

9. Le résultat de l'exécution de la commande « ***docker service ls*** » sur la machine A est représenté dans l'affichage ci-dessous. Interpréter ce résultat. (0.75 pt)

```
ID             NAME             MODE         REPLICAS   IMAGE                PORTS
7m5z8z9jv8x6   myapp_frontend   replicated   1/1        my-frontend          *:3001->3001/tcp
i7kp519cv8v5   myapp_backend    replicated   1/1        my-backend           *:3000->3000/tcp
2jb9z916h7w5   myapp_database   replicated   1/1        postgres:13-alpine   *:27017->5432/tcp
```

L'application myapp contient ***3 services*** *(myapp_frontend, myapp_backend et myapp_database) (0.25pt)→ nommer l'image pour chaque services (0.25pt)*
Chaque service est d'un seul réplica (0.25pt)

10. On souhaite mettre en échelle le service « *myapp_frontend* » avec un nombre des replicas=3. Donner la commande correspondante. (0.5 pt)

docker service scale myapp_frontend=3

11. Dans l'objectif de surveiller l'état des services du « Stack », nous avons obtenu cet exemple d'affichage. Interpréter ce résultat. (0.5 pt)

```
ID             NAME                 IMAGE        NODE           DESIRED STATE   CURRENT STATE           ERROR     PORTS
19kp4v8c5x6h   myapp_frontend.1     my-frontend  manager-node   Running         Running 5 minutes ago
o3jb9z4v812h   myapp_frontend.2     my-frontend  worker-node-1  Running         Running 5 minutes ago
q7kp613cv4v2   myapp_frontend.3     my-frontend  worker-node-2  Running         Running 5 minutes ago
```

Le service myapp_frontend est de 3 réplicas actifs sur les 3 machines (machine A (master), nommé manager-node, machine B et C (worker-node 1 et 2))

12.Pour assurer la communication entre les conteneurs de différents nœuds du cluster, quel type de réseau est utilisé ? (0.25 pt)

Overlay